

Pentair API V3 Single Property

Notes

- All payloads are JSON
- All messages contain data from a single property managed by a web socket connection to that property context.

Return Codes

Code	Description
200	OK
404	Not Found
500	Internal Server Error
201	Created
304	Not Modified
400	Bad Request
401	Unauthorized
403	Forbidden
416	Out of Range

Response message

All client requests will be responded to with a response message containing the message Id of the request and a response code from the above list of return codes. The response message is one of two types, messages that have responses to them such as GetParamList, RequestParamList, etc will contain the response code in the responding message. Messages that do not contain specific responses such as SetParamList, ClearParamList, etc will be responded to by echoing the message header and include the response code.

GetParamList / SendParamList

GetParamList is used to retrieve information about the target hardware. The message can contain specific requests about one or more objects or general queries about system status. The response from the target hardware is the SendParamList message.

NOTES:

1. Ordering the output is performed with the keywords 'INCR' and 'DECR' for the objname value. Ordering is performed on the first parameter listed in the keys list, in the second example the output would be ordered in ascending order of the value for LISTORD.
2. The allowable operators used in the condition clause are listed in the table below.

GetParamList	SendParamList
<pre>{ "command": "GetParamList", "messageID": "10A86ef1-88f1-8091-7cdf19", "condition": "", "objectList": [{ "objnam": "C1055", "keys": ["STATUS", "TIMOUT"] }] }</pre>	<pre>{ "command": "SendParamList", "messageID": "10A86ef1-88f1-8091-7cdf19", "response": "200", "objectList": [{ "objnam": "C1055", "params": { "STATUS": "ON", "TIMOUT": "3600" } }] }</pre>
<pre>{ "command": "GetParamList", "messageID": "10A86ef1-88f1-8091-7cdf1a", "condition": "OBJTYP=CIRCUIT & SUBTYP!GENERIC ", "objectList": [{ "objnam": "INCR", "keys": ["LISTORD", "SNAME", "STATUS"] }] }</pre>	<pre>{ "command": "SendParamList", "messageID": "10A86ef1-88f1-8091-7cdf1a", "response": "200", "objectList": [{ "objnam": "C1055", "params": { "LISTORD": "1", "SNAME": "Pool Light", "STATUS": "ON" } }, { "objnam": "C1056", "params": { "LISTORD": "2", "SNAME": "Spa Light", "STATUS": "ON" } }] }</pre>

Condition operators

Operator	Description
=	Equality
!	Inequality
&	Logical AND
	Logical OR
()	Parenthesis to enforce proper operator order
*	Contains

GetQuery / SendQuery

GetQuery is used to retrieve information about the target hardware. The message can contain specific requests about one or more objects or general queries about system status. The response from the target hardware is the SendQuery message.

NOTE: Optional arguments are provided as a list of key / value pairs.

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetConfiguration", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetConfiguration", "response": "200", "answer": "<agreed upon format>" }</pre>

SetCommand

SetCommand provides an abstracted method for performing low level tasks on the target hardware.

NOTE: optional arguments are provided as a list of key / value pairs, see specific commands as detailed in Predefined Commands below.

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetBodyState", "arguments": { "B1000": "ON" } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetBodyState", "response": "200" }</pre>

RequestParamList

Like the GetParamList, the RequestParamList is used to retrieve information about the target hardware. The RequestParamList can not contain queries (conditions). The NotifyList provides the response to the RequestParamList, the initial NotifyList is populated with all requested parameter values, subsequent changes to any of these values will generate a new NotifyList message containing only the key value pairs that have changed. Note: only the initial NotifyList response will contain a response code and messageID.

NOTES:

1. The NotifyList message will only contain a messageID when responding to the RequestParamList, all NotifyList messages is considered a broadcast to any client listening to the addressed hardware.
2. When a client detects that a NotifyList message has not been received using the timeSince property value, it will issue a ClearParam message followed by a new RequestParamList message to obtain a complete update of values that it has subscribed to.
3. NotifyList messages will only contain a response code when responding to the RequestParamList, all NotifyList messages is considered a broadcast to any client listening to the addressed hardware

RequestParamList	NotifyList
<pre>{ "command": "RequestParamList", "messageID": "10A86ef1-88f1-8091-7cdf1c", "objectList": [{ "objnam": "C1055", "keys": ["STATUS"] }, { "objnam": "C1056", "keys": ["STATUS"] }] }</pre>	<pre>{ "command": "NotifyList", "response": "200", "messageID": "10A86ef1-88f1-8091-7cdf1c", "timeNow": "123456789", "timeSince": "123456788", "objectList": [{ "objnam": "C1055", "params": { "STATUS": "ON" } }, { "objnam": "C1056", "params": { "STATUS": "ON" } }] }</pre>

ReleaseParamList

ReleasePareamList cancels previously scheduled subscriptions to parameter changes made with the RequestParamList message. The response to this is a ReleaseParamList with the message Id and a response code.

ReleaseParamList	Response
<pre>{ "command": "ReleaseParamList", "messageID": "10A86ef1-88f1-8091-7cdf1d", "objectList": [{ "objnam": "C1055", "keys": ["STATUS"] }, { "objnam": "C1056", "keys": ["STATUS"] }] }</pre>	<pre>{ "command": "ReleaseParamList", "messageID": "10A86ef1-88f1-8091-7cdf1d", "response": "200" }</pre>

ClearParam

ClearParam cancels all previously scheduled subscriptions to parameter changes made with any prior RequestParamList message. The response to tis is a ClearParam with the message Id and a response code.

ClearParam	Response
<pre>{ "command": "ClearParam", "messageID": "10A86ef1-88f1-8091-7cdf1e" }</pre>	<pre>{ "command": "ClearParam", "messageID": "10A86ef1-88f1-8091-7cdf1e", "response": "200" }</pre>

SetParamList

SetParamList is used to issue commands on the target hardware. The message contains one or more commands that the target hardware is to carry out. The SetParamList has no specific

response message, the target hardware will acknowledge the SetParamList with a SetParamList response that includes a response code.

SetParamList	Response
<pre>{ "command": "SetParamList", "messageID": "10A86ef1-88f1-8091-7cdf1f", "objectList": [{ "objnam": "C1055", "params": { "STATUS": "ON", "TIMEOUT": "3600" } }, { "objnam": "C1065", "params": { "ACT": "ROYAL" } }] }</pre>	<pre>{ "command": "SetParamList", "messageID": "10A86ef1-88f1-8091-7cdf1f", "response": "200" }</pre>

ReadStatusMessage / WriteStatusMessage

ReadStatusMessage is used to retrieve missing status messages when the condition is detected. The WriteStatusMessage is sent from the target hardware whenever a change is detected in a status of the equipment managed by the target hardware. It is also send on demand by a ReadStatusMessage request.

NOTES:

- 1. ReadStatusMessage is sent by any client on first attaching to the target hardware in local or remote mode, a value of 0 will be presented as the timeSince value to obtain a current set of status conditions.
- 2. The WriteStatusMessage contains a messageID only when responding to the ReadStatusMessage. All WriteStatusMessage messages are considered a broadcast to anyone listening to the addressed target hardware.
- 3. WriteStatusMessage contains a response code only when responding to a ReadStatusMessage request.

IMPORTANT: The 'created' node which lists all newly created alert and warning object names is optional. By definition any alert or warning object that has been created since the last synchronization event is a change, the newly created object must be included in the 'changes' list along with all of its parameters in order for it to be processed by any client.

ReadStatusMessage	WriteStatusMessage
<pre>{ "command": "ReadStatusMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "timeSince": "123456789" }</pre>	<pre>{ "command": "WriteStatusMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "response": "200", "timeNow": "123456789", "timeSince": "123456788", "objectList": [{ "deleted": ["G1000", "G1001"], "created": ["G3221"], "changes": [{ "objnam": "G3221", "params": { "OBJTYP": "STAMSG", "PARENT": "I1002", "MODE": "LTRED", "SNAME": "Heater is cooling down", "SINDEX": "PHLO" } }] }] }</pre>

ReadHistoryMessage / WriteHistoryMessage / ErrorHistoryMessage

2017-04-06 VB: Server will not send ReadHistoryMessage request anymore. See ReadLastHistoryTime/WriteLastHistoryTime below. ErrorHistoryMessage added.

ReadHistoryMessage is sent by client to retrieve history records for the specified time interval. The timeEnd value will define the ending time for any status change that occurs, a zero (0) means that the server should send all statuses up to the current point in time.

WriteHistoryMessage is sent from the target hardware on first attaching to the server, or whenever a change is detected in a status of the equipment managed by the target hardware for which history is maintained. THW assigns unique messageID to each message. It also sent from the server in response to client's ReadHistoryMessage request. In this case all the messages in a "session" should copy the request messageID and should contain the "response" status code.

ErrorHistoryMessage is sent by the server if the WriteHistoryMessage processing failed. The messageNo and the timestamp values shall match those from the failed WriteHistoryMessage request. If the request cannot be deserialized, the messageNo shall be "-1" and the timestamp shall be assigned to the value of the last successfully saved history record (or "-1", if database reading error).

ReadHistoryMessage	WriteHistoryMessage	ErrorHistoryMessage
<pre>{ "command": "ReadHistoryMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "timeStart": "123456789", </pre>	<pre>{ "command": "WriteHistoryMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "response": "400", </pre>	<pre>{ "command": "ErrorHistoryMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "response": "400", </pre>

<pre>"timeEnd": "0" }</pre>	<pre>"messageNo": "30", "timeNow": "123456789", "timeSince": "123456789", "objectList": [{ "C1000": [{ "P": "STATUS", "V": "ON", "T": "12332" }, { "P": "STATUS", "V": "ON", "T": "12302" }], "P1000": [{ "P": "RPM", "V": "1200", "T": "12328" }, { "P": "RPM", "V": "950", "T": "12332" }, { "P": "RPM", "V": "200", "T": "12332" }, { "P": "RPM", "V": "5", "T": "12332" }, { "P": "STATUS", "V": "ON", "T": "12332" }] }]</pre>	<pre>"messageNo": "30", "timestamp": "123456789", "description": "Value was either too large or too small for a UInt32" }</pre>

ReadLastHistoryTime / WriteLastHistoryTime

2017-04-06 VB: New messages added.

ReadLastHistoryTime is sent by THW to the server to retrieve timestamp of the last WriteHistoryMessage received from the THW device. The server responds with the WriteLastHistoryTime.

ReadLastHistoryTime use case list is including, but not limited to sending it to the server on first attaching to the target hardware, or when resuming the history posting "session" after it was interrupted by error or network problem.

CleanHistoryMessage is sent by THW to the server when all or a part of history data need to remove (delete) from the system. The parameter timeStart and timeEnd described the range of the data. The zero (0) value means the begin or end the data rang.

ReadLastHistoryTime	WriteLastHistoryTime	CleanHistoryMessage
<pre>{ "command": "ReadLastHistoryTime", "messageID": "10A86ef1-88f1-8091-7cdf21" }</pre>	<pre>{ "command": "WriteLastHistoryTime", "messageID": "10A86ef1-88f1-8091-7cdf21", "response": "200", "lastHistoryTime": "123456789" }</pre>	<pre>{ "command": "CleanHistoryMessage", "messageID": "10A86ef1-88f1-8091-7cdf21", "timeStart": "123456789", "timeEnd": "0" }</pre>

GetRegisteredProperties

GetRegisteredProerties is used by the target hardware to refresh an existing registration with the Pentair server. When obtaining a web socket connection with the Pentair server, an authentication process is performed during which the target hardware presents its unique ID and a shared secret. If the target hardware has lost either of these credentials it can no longer open a connection with the Pentair server. The GetRegisteredProperties message returns a list of properties associated with the user credentials supplied in the request. The user can then select the target hardware's property from the list and re-register the target hardware using the RequestRegistration message.

GetRegisteredProperties	GetRegisteredProperties
<pre>{ "command": "GetRegisteredProperties", "messageID": "10A86ef1-88f1-8091-7cdf22", "userID": "email@pentair.com", "password": "password" }</pre>	<pre>{ "command": "GetRegisteredProperties", "messageID": "10A86ef1-88f1-8091-7cdf22", "response": "200", "objectList": [{ "InstallationID": 1, "PoolName": "Florida vacation home", "Address": "123 Palm Beach Avenue", "City": "West Palm Beach", "State": "Florida", "Zip": "30202", "OnLine": "NO" }, { "InstallationID": 5, "PoolName": "Colorado home", "Address": "123 Gold Blvd", "City": "Pueblo", "State": "Colorado", "Zip": "76231", "OnLine": "YES" }] }</pre>

Predefined Queries (GetQuery methods)

Stored procedures are static objects that represent a set of objects and specific property values contained in those objects. It provides a means for an authenticated client to querying the target hardware in local or remote mode to obtain information about the target hardware at more abstract level than the query language built into the GetParamList / SendParamList messaging scheme.

Query name	Description	Simulated / Implemented
GetConfiguration	All data necessary to display the Dashboard, each Body and all Chemistry screens (Deprecated)	Yes
GetSystemConfiguration	All configuration data required for the dashboard and main screens	No
GetUserSettings	Return the property owner, location, preference and time settings	No
GetStatusMessageFlags	Return the Status Message Flags by object type.	Yes
GetActiveStatusMessages	Return all active Status, Alert and Warning messages	Yes
GetValidLightGroupings	Return all Light Circuits that can be grouped together	Yes
GetHardwareDefinition	Return the hardware configuration	Yes
AddressMap	Return list of available addresses for Comm Addressable devices	No
GetRemotes	Return list of available remotes	No
GetCircuitNames	Return a list of suggested circuit names	Yes
GetCircuitTypes	Return a list of a circuit's subtypes system name and readable name	Yes

Predefined Commands (SetCommand methods)

The following commands provide an abstracted method of setting the current state for objects or object groups on the target hardware.

Method	Arguments	Description	Simulated / Implemented
SetBodyState	OBJNAM: <ON/OFF>	Turn body of water on / off	Yes
MasterOff	No parameters	Turn all circuits off (delayed turn off and freeze overrides apply)	Yes
EditCircuitGroup	See description	Changes the collection of circuits assigned to and / or renames the group of lights or generic circuits. (Deprecated)	Yes
EditGroup	See description	Changes the collection of assigned objects and / or renames the group of lights, generic circuits or heater combinations and their properties.	No
SetStatusMessageFlags	See description	Set the Status Message Flags for the given object type	Yes
ClearStatusMessages	See description	Clear displaying Warning and Alert messages.	Yes
EditObjectAffect	See description	Edit Pump and its affects	No

EditRemote	See description	Edit Remote	No

EditCircuitGroup

The command EditLightGroup modifies an existing light group. This command allows the displayed name (SNAME) to be modified as well as the collection of circuits represented by this circuit group. The command accepts a list of OBJNAM objects which are objects of type CIRCUIT and which represent lights of any type or generic circuits. If the group being modified is of type LITSHO, the THW will modify the USAGE parameter of the parent circuit group with the appropriate Light Mode Enumerations based on the types of lights contained in the group

Example:

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditCircuitGroup", "arguments": { "OBJNAM": "C1006", "OBJLIST": [{ "objnam": "C1023", "params": { "ACT": "ON", "LISTORD": "1" } }, { "objnam": "C1024", "params": { "ACT": "ON", "LISTORD": "1" } }, { "objnam": "C2013", "params": { "ACT": "ON", "LISTORD": "1", } }], "SNAME": "East Wing Romance", "SHOMNU": "1", "COOL": "OFF", "FREEZE": "OFF" } }</pre> Add Parent	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditCircuitGroup", "response": "200", "objectList": [{ "OBJNAM": "C8322", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "CIRCGRP / LITSHO", "SNAME": "East Wing Romance", "LISTORD": "1", "SHOMNU": "1", "USAGE": "rpcsQdyMbaSowglTeiGnqBARL", "COUNT": "3", "OBJLIST": [{ "objnam": "c0001", "params": { "OBJTYP": "CIRCGRP / LITSHO", "CIRCUIT": "C1023", "ACT": "", "DLY": "0", "LISTORD": "1", "COLOR": "", "START": "1" } }, { "objnam": "c0002", "params": { "OBJTYP": "CIRCGRP / LITSHO", "CIRCUIT": "C1024", "ACT": "", "DLY": "0", "LISTORD": "2", "COLOR": "", "START": "1" } }, { "objnam": "c0003", "params": { "OBJTYP": "CIRCGRP / LITSHO", "CIRCUIT": "C2013", "ACT": "", "DLY": "0", "LISTORD": "3", "COLOR": "", "START": "1" } }] } }] }</pre>

Response Codes:

--	--

Code	Definition
200	OK, A status message may be generated if the action results in a DLYON or DLYOFF state
400	Bad request, unable to process message contents.
403	Forbidden for any reason, eg: service mode, etc.
404	Not found, one of the circuits in the arguments list was not found.

EditGroup

The command CreateGroup replaces and deprecates the CreateCircuitGroup command, adding support for heater combination groups. It creates a CIRCUIT of subtype CIRCGRP or LITSHO or a heater of subtype HTCOMBO depending on the OBJTYP parameter passed in. Each circuit or heater specified in the OBJLIST will be linked with a new affect object and the parameters specified by that affect. The THW will be responsible for providing the following information in the response based on the group being created:

- 1. For groups of type LITSHO, the target hardware will populate the USAGE parameter of the parent circuit group with the appropriate Light Mode Enumerations based on the types of lights contained in the group.
- 2. For groups of type HTCOMBO, the THW will populate the SHOMNU with the capabilities of the heater group.

The response will include the new group OBJNAM that was created along with an objectList of child groups that were created.

Example:

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditGroup", "arguments": { "OBJNAM": "C8322", "OBJLIST": [{ "objnam": "C1023", "params": { "ACT": "ON", "LISTORD": "1" } }, { "objnam": "C1024", "params": { "ACT": "ON", "LISTORD": "1" } }, { "objnam": "C2013", "params": { "ACT": "ON", "LISTORD": "1" } }], "SNAME": "East Wing Romance" } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditGroup", "response": "200", "objectList": [{ "OBJNAM": "C8322", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "CIRCGRP", "SNAME": "East Wing Romance", "LISTORD": "1", "SHOMNU": "1", "USAGE": "rpcsQdyMbaSowglTeiGnqBARL", "COUNT": "3", "OBJLIST": [{ "objnam": "c0001", "params": { "OBJTYP": "CIRCGRP", "CIRCUIT": "C1023", "ACT": "", "DLY": "0", "LISTORD": "1", "START": "1" } }, { "objnam": "c0002", "params": { "OBJTYP": "CIRCGRP", "CIRCUIT": "C1024", "ACT": "", "DLY": "0", "LISTORD": "2", "START": "1" } }, { "objnam": "c0003", "params": { "OBJTYP": "CIRCGRP / LITSHO", "CIRCUIT": "C2013", "ACT": "", "DLY": "0", "LISTORD": "3", "START": "1" } }] } }] }</pre>

Group Parameters	Circuit Affect	Circuit Affect Parameters	Value
OBJNAM			CIRCGRP
SNAME			User assigned name
OBJLIST	objnam		Circuit being linked
	params	ACT	ON / OFF

Light Show

Group Parameters	Circuit Affect	Circuit Affect Parameters	Value
OBJNAM			LITSHO
SNAME			User assigned name
OBJLIST	objnam		Circuit being linked
	params		null

Heater Combo

Group Parameters	Circuit Affect	Circuit Affect Parameters	Value
OBJNAM			HCOMBO
SNAME			User assigned name
ACT			CONCUR / PRIOR
OBJLIST	objnam		Circuit being linked
	params	LISTORD	1

Response Codes:

Code	Definition
200	OK, A status message may be generated if the action results in a DLYON or DLYOFF state
400	Bad request, unable to process message contents.
403	Forbidden for any reason, eg: service mode, etc.
404	Not found, one of the circuits in the arguments list was not found.

Edit Object Affect

The command EditObjectAffect allows an existing affect to be changed. Valid object affects include VLVCIIRC, PMPCIRC and HCOMBO. The response will contain the same payload as the query for each of the object types (GetValveConfiguration, GetPumpConfiguration, GetHeaterConfiguration) with header schema as defined below.
Example (VLVCIRC):

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditObjectAffect", "arguments": { "OBJNAM": "V1000", "objectList": [{ "CIRCUIT": "C1000", "POSITION": "1", "PRIOR": "2" }, { "CIRCUIT": "C1001", "POSITION": "2", "PRIOR": "1" }] } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditObjectAffect", "response": "200", "objectList": [{ "objnam": "V1000", "params": { "SUBTYP": "LEGACY / INTELLI", "SNAME": "RETURN", "COMUART": "1", "POSIT": "2", "ASSIGN": "INTAKE", "DLY": "ON", "OBJLIST": [{ "objnam": "v1000", "params": { "POSIT": "2", "PRIOR": "1", "CIRUIT": "C1000" } }] } }, { "objnam": "V1001", "params": { "SUBTYP": "LEGACY / INTELLI", "SNAME": "RETURN", "COMUART": "1", "POSIT": "2", "ASSIGN": "RETURN", "DLY": "ON", "OBJLIST": [{ "objnam": "v1001", "params": { "POSIT": "2", "PRIOR": "1", "CIRUIT": "C1010" } }] } }] }</pre>

Example (PMPCIRC):

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditObjectAffect", "arguments": { "OBJNAM": "P1000", "objectList": [{ "CIRCUIT": "C1000", "LISTORD": "1", "SPEED": "1", "BOOST": "ON", "SELECT": "GPM" }, { "CIRCUIT": "C1000", "LISTORD": "2", "SPEED": "2", "BOOST": "ON", "SELECT": "GPM" }] } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "EditObjectAffect", "response": "200", "objectList": [{ "objnam": "P1000", "params": { "SUBTYP": "SINGLE / DUAL / SPEED / FLOW", "SNAME": "RETURN", "COMUART": "1", "BODY": "B1000", "SHARE": "B1002", "DLY": "ON", "OBJLIST": [{ "objnam": "p1000", "params": { "CIRUIT": "C1000", "LISTORD": "1", "SPEED": "100", "BOOST": "OFF", "SELECT": "GPM" } }] } }, { "objnam": "V1001", "params": { "SUBTYP": "INTELLI", "SNAME": "RETURN", "ASSIGN": "RETURN", "DLY": "ON", "OBJLIST": [{ "objnam": "v1001", "params": { "CIRUIT": "C1010", "LISTORD": "1", "SPEED": "100", "BOOST": "OFF", "SELECT": "GPM" } }] } }] }</pre>

Response Codes:

Code	Definition
------	------------

200	OK, A status message may be generated if the action results in a DLYON or DLYOFF state
400	Bad request, unable to process message contents.
403	Forbidden for any reason, eg: service mode, etc.
404	Not found, one of the circuits in the arguments list was not found.

Edit Remote

The EditRemotes command sends a new definition for one of the remotes contained on the THW. Those remotes that the command does contain will include a complete definition of all editable parameters. The response to this command contains the same payload as the GetRemotes query, returning all remotes, their parameters and collection of child button objects.

The IS10 and APACMD remotes support a programming feature that allows their communications address to be changed. This functionality involves turning the MODE parameter ON, assigning the new COMUART address and then turning the MODE parameter back to the OFF state. The client is responsible for signaling this set of state transitions to occur by supplying the proper COMUART address and setting the MODE parameter ON in the SetCommand message. The THW will be responsible for performing the various state transitions to effect the programming feature before responding with the new COMUART address and the MODE parameter back in the OFF state.

Example (list may include multiple remotes):

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf1b", "method": "EditRemote", "arguments": [{ "objnam": "R1000", "params": { "SNAME": "Pool Remote", "COMUART": "1", "LISTORD": "1", "BODY": "B1000", "PUMP": "P1000", "ENABLE": "ON", "BUTTONS": [{ "OBJNAM": "b1000", "LISTORD": "1", "CIRCUIT": "C1200" }, { "OBJNAM": "b1002", "LISTORD": "2", "CIRCUIT": "C1202" }, { "OBJNAM": "b1003", "LISTORD": "3", "PARENT": "R1000", "CIRCUIT": "00000" }, { "OBJNAM": "b1004", "LISTORD": "4", "CIRCUIT": "00000" }] } }] }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf1b", "method": "EditRemote", "response": "200", "answer": [{ "objnam": "R1000", "params": { "SUBTYP": "IS4", "SNAME": "Pool Remote", "COMUART": "1", "LISTORD": "1", "BODY": "B1000", "PUMP": "P1000", "ENABLE": "ON", "MODE": "ON", "BUTTONS": [{ "OBJNAM": "b1000", "LISTORD": "1", "PARENT": "R1000", "CIRCUIT": "C1200" }, { "OBJNAM": "b1002", "LISTORD": "2", "PARENT": "R1000", "CIRCUIT": "C1202" }, { "OBJNAM": "b1003", "LISTORD": "3", "PARENT": "R1000", "CIRCUIT": "00000" }, { "OBJNAM": "b1004", "LISTORD": "4", "PARENT": "R1000", "CIRCUIT": "00000" }] } }] }</pre>

Support Parameters:

	IS4	IS10	SPACMD	QT
BODY		X		
PUMP				

ENABLE	X	X	X	X
COMUART		X	X	

GetActiveStatusMessages

The GetActiveStatusMessages query returns all Status, Alert and Warning messages that are currently active on the property. The query returns a list of all currently active STAMSG objects and the timeSince value of the last update to any object represented by the TIME parameter on the _57A7 singleton object. This message will be used to obtain the list of Status, Alert and Warning messages when a client starts a session with a property, all subsequent Status, Alert and Warning messages will be received during the lifetime of a client session by processing WriteStatusMessage broadcasts and using the ReadStatusMessage functionality when a missing message is detected.

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf21", "queryName": "GetActiveStatusMessages", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf21", "queryName": "GetActiveStatusMessages", "response": "200", "timeSince": "123456788", "answer": [{ "objnam": "G3221", "params": { "OBJTYP": "STAMSG", "PARENT": "H1000", "MODE": "STATUS", "SNAME": "Heater is cooling down", "SINDEX": "DLY", "SHOMNU": "ON" } }, { "objnam": "G3222", "params": { "OBJTYP": "STAMSG", "PARENT": "I1002", "MODE": "LTRED", "SNAME": "Heater is cooling down", "SINDEX": "PHLO", "SHOMNU": "ON" } }] }</pre>

Get Circuit Names

The GetCircuitNameSuggestions query returns a list of suggested circuit names that can be used when creating or renaming circuits on the THW. This list is presented to the user as a drop down list of recommendations when selecting a circuit name. The client will present the list in a drop down selector in the order it is received from the THW.

Example:

GetQuery	SendQuery
GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetCircuitNames", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetCircuitNames", "response": "200", "answer": ["AERATOR", "AIR BLOWER", "BACKWASH", "BACK LIGHT", "BBQ LIGHT", "BEACH LITE", "BENCH", "BLOWER", "JETS", "MUSIC", "SLIDE", "SPA", "SPA HIGH", "SPA LIGHT", "SPA LOW", "STREAM", "SWIM JETS", "WATERFALL", "YARD LIGHT"] }</pre>

Get Circuit Types

The GetCircuitNameSuggestions query returns a list of suggested circuit names that can be used when creating or renaming circuits on the THW. This list is presented to the user as a drop down list of recommendations when selecting a circuit name. The client will present the list in a drop down selector in the order it is received from the THW.

Example:

GetQuery	SendQuery
GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetCircuitTypes", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetCircuitTypes", "response": "200", "answer": [{ "systemValue": "GENERIC", "readableValue": "Generic" }, { "systemValue": "SPILL", "readableValue": "Spillway" }, { "systemValue": "MASTER", "readableValue": "Master" }, { "systemValue": "CHEM", "readableValue": "Chem" }, { "systemValue": "LIGHT", "readableValue": "Light" }, { "systemValue": "INTELLI", "readableValue": "IntelliBrite" }, { "systemValue": "GLOW", "readableValue": "GloBrite" }, { "systemValue": "GLOWT", "readableValue": "GloBrite" }, { "systemValue": "MAGIC1", "readableValue": "Magic" }, { "systemValue": "MAGIC2", "readableValue": "Magic" }, { "systemValue": "SAML", "readableValue": "SAm/SAL" }, { "systemValue": "DIMMER", "readableValue": "Dimmer" }, { "systemValue": "CLRCASC", "readableValue": "ColorCascade" }, { "systemValue": "PHOTON", </pre>

```
"readableValue": "PG2000"  
}  
]  
}
```

GetConfiguration

The GetConfiguration query returns all configuration (Saved / Hard Coded) data necessary to display the home page. This includes all bodies, lights, light groups, features, heaters, and chemistry. The client will use this information to subscribe to Live parameters based on the screen being displayed. Items that are not included in this message include any circuits that are displayed only in configuration screens, orphaned heaters, chemistry and circuit affects.

Changes:

- 1. The PARENT property was added to all CIRCGRP and LITSHO objects
- 2. The CHILD property was added to all CIRCUIT objects
- 3. A Heater Combo has been added to the body circuit for clarification

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetConfiguration", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetConfiguration", "response": "200", "answer": [{ "objnam": "UFFFFE", "params": { "ENABLE": "ON" } }, { "objnam": "_C10C", "params": { "CLK24A": "AMPM" } }, { "objnam": "_5451", "params": { "MODE": "ENGLISH" } }, { "objnam": "B0001", "params": { "OBJTYP": "BODY", "SUBTYP": "POOL", "SNAME": "Lower Pool", "LISTORD": "1", "HITMP": "82", "LOTMP": "78", "HTSRC": "H0001", "SHARE": "B1002", "PRIME": "C1002", "SEC": "C1003", "ACT1": "C1005", "ACT2": "C1008", "ACT3": "C1012", "ACT4": "C1A3B", "OBJLIST": [{ "objnam": "H0001", "params": { "OBJTYP": "HEATER", "SUBTYP": "HTPMP", "SNAME": "Heat Pump", "LISTORD": "1" } }, { "objnam": "H0002", "params": { "OBJTYP": "HEATER", "SUBTYP": "SOLAR", } }] } }] }</pre>

```

        "SNAME": "SolarPanel",
        "LISTORD": "2"
    },
    {
        "objnam": "H0002",
        "params": {
            "OBJTYP": "HCOMBO",
            "SUBTYP": "SOLAR",
            "SNAME": "Solar preferred",
            "LISTORD": "3",
            "OBJLIST": [
                {
                    "objnam": "H0002",
                    "params": {
                        "OBJTYP": "HEATER",
                        "SUBTYP": "SOLAR",
                        "SNAME": "SolarPanel",
                        "LISTORD": "1"
                    }
                },
                {
                    "objnam": "H0001",
                    "params": {
                        "OBJTYP": "HEATER",
                        "SUBTYP": "HTPMP",
                        "SNAME": "Heat Pump",
                        "LISTORD": "2"
                    }
                }
            ]
        }
    },
    {
        "objnam": "I1000",
        "params": {
            "OBJTYP": "CHEM",
            "SUBTYP": "ICHLOR",
            "SNAME": "PoolClorinator"
        }
    },
    {
        "objnam": "I1001",
        "params": {
            "OBJTYP": "CHEM",
            "SUBTYP": "ICHEM",
            "SNAME": "Chemistry"
        }
    }
]
},
{
    "objnam": "C1002",
    "params": {
        "OBJTYP": "CIRCUIT",
        "SUBTYP": "MAGIC2",
        "SNAME": "PoolLight",
        "LISTORD": "1",
        "SHOMNU": "f1",
        "CHILD": "c1001, c1002",
        "USE": "OFF",
        "USAGE": "RPCSqDYmBASOWGitEIgNQbar1",
        "OBJLIST": [
            {
                "objnam": "c1000",
                "params": {
                    "OBJTYP": "CIRCGRP",
                    "CIRCUIT": "C1002",
                    "PARENT": "C1000",
                    "ACT": "OFF",
                    "LISTORD": "2",
                    "DLY": "5"
                }
            },
            {
                "objnam": "c1002",
                "params": {
                    "OBJTYP": "LITSHO",
                    "CIRCUIT": "C1002",
                    "PARENT": "C1000",
                    "ACT": "OFF",
                    "LISTORD": "2",
                    "DLY": "10"
                }
            }
        ]
    }
}

```

--	--

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

GetHardwareDefinition

The GetHardwareDefinition query returns the hardware configuration in a Panel / Module centric view. This definition includes the identity and type of PANEL, MODULE and relay CIRCUIT objects contained in the hardware configuration along with all body objects, heater and heat combo objects, pump and pump affect objects, valve and valve affect objects.

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetHardwareDefinition", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetHardwareDefinition", "response": "200", "answer": [{ "objnam": "M0001", "params": { "OBJTYP": "PANEL", "SUBTYP": "OCP", "HNAME": "PANEL", "SNAME": "Lower Pool", "LISTORD": "1", "VER": "1", "OBJLIST": [{ "objnam": "m0001", "params": { "OBJTYP": "MODULE", "SUBTYP": "I5P", "SNAME": "I5P", "LISTORD": "2", "PARENT": "M0001", "PORT": "2", "VER": "1", "BADGE": "OFF", "CIRCUITS": [{ "objnam": "C1000", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "GENERIC", "HNAME": "AUX1", "SNAME": "", "LISTORD": "", "PARENT": "m0001", "BODY": "", "FREEZE": "OFF", "VER": "1", "TIMZON": "120", "TIME": "120", "RLY": "r1000", "SVC": "OFF", "DNTSTP": "OFF", "FEATR": "OFF" } }, { "objnam": "C1001", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "GENERIC", "HNAME": "AUX2", "SNAME": "", "LISTORD": "", "PARENT": "m0001", "BODY": "", "FREEZE": "OFF", "VER": "1", "TIMZON": "120", "TIME": "120", "RLY": "r1000", "SVC": "OFF", "DNTSTP": "OFF", "FEATR": "OFF" } }] } }, { "objnam": "B1000", "params": { "OBJTYP": "BODY", "SUBTYP": "POOL", "SNAME": "Lower Pool", "LISTORD": "1", "HITMP": "82", } }] } }] }</pre>

```

        "LOTMP": "78",
        "HTSRC": "H0001",
        "SHARE": "B1002",
        "PRIME": "C1002",
        "SEC": "C1003",
        "ACT1": "C1005",
        "ACT2": "C1008",
        "ACT3": "C1012",
        "ACT4": "C1A3B",
        "VOL": "",
        "MANUAL": "OFF",
        "MANHT": ""
    }
},
{
    "objnam": "H0002",
    "params": {
        "OBJTYP": "HCOMBO",
        "SUBTYP": "SOLAR",
        "SNAME": "Solar preferred",
        "LISTORD": "3",
        "OBJLIST": [
            {
                "objnam": "H0002",
                "params": {
                    "OBJTYP": "HEATER",
                    "SUBTYP": "SOLAR",
                    "SNAME": "SolarPanel",
                    "LISTORD": "1"
                }
            },
            {
                "objnam": "H0001",
                "params": {
                    "OBJTYP": "HEATER",
                    "SUBTYP": "HTPMP",
                    "SNAME": "Heat Pump",
                    "LISTORD": "2"
                }
            }
        ]
    }
},
{
    "objnam": "P1000",
    "params": {
        "SUBTYP": "SINGLE / DUAL / SPEED / FLOW",
        "SNAME": "RETURN",
        "COMUART": "1",
        "PRIMFLO": "2000",
        "PRIMTIM": "2",
        "SYSTIM": "3",
        "MIN": "300",
        "MAX": "30000",
        "PRIOR": "SPEED | RECENT | CUSTOM",
        "BODY": "B1000",
        "SHARE": "B1002",
        "DLY": "ON",
        "SETTMP": "2",
        "OBJLIST": [
            {
                "objnam": "p1000",
                "params": {
                    "CIRCUIT": "C1000",
                    "LISTORD": "1",
                    "SPEED": "100",
                    "BOOST": "OFF",
                    "SELECT": "GPM"
                }
            },
            {
                "objnam": "p1001",
                "params": {
                    "CIRCUIT": "C1000",
                    "LISTORD": "1",
                    "SPEED": "100",
                    "BOOST": "OFF",
                    "SELECT": "GPM"
                }
            }
        ]
    }
},
{
    "objnam": "V1000",
    "params": {
        "SUBTYP": "LEGACY / INTELLI",
        "SNAME": "RETURN",
        "COMUART": "1",
        "POSIT": "2",
        "ASSIGN": "INTAKE",
        "DLY": "ON",
        "OBJLIST": [
            {
                "objnam": "v1000",
                "params": {

```

```

        "POSIT": "2",
        "PRIOR": "1",
        "CIRUIT": "C1000"
    }
}

    ]
}

    }
}

    ]
}

    }
}

    "objnam": "m0002",
    "params": {
        "OBJTYP": "MODULE",
        "SUBTYP": "I5X",
        "SNAME": "I5X",
        "LISTORD": "1",
        "PARENT": "M0001",
        "PORT": "2",
        "VER": "1",
        "BADGE": "OFF",
        "CIRCUITS": [
            {
                "objnam": "C1002",
                "params": {
                    "OBJTYP": "CIRCUIT",
                    "SUBTYP": "GENERIC",
                    "HNAME": "AUX5",
                    "SNAME": "",
                    "LISTORD": "",
                    "PARENT": "m0002",
                    "BODY": "",
                    "FREEZE": "OFF",
                    "VER": "1",
                    "TIMZON": "120",
                    "TIME": "120",
                    "RLY": "x1000",
                    "SVC": "OFF",
                    "DNTSTP": "OFF",
                    "FEATR": "OFF"
                }
            },
            {
                "objnam": "C1003",
                "params": {
                    "OBJTYP": "CIRCUIT",
                    "SUBTYP": "GENERIC",
                    "HNAME": "AUX6",
                    "SNAME": "",
                    "LISTORD": "",
                    "PARENT": "m0002",
                    "BODY": "",
                    "FREEZE": "OFF",
                    "VER": "1",
                    "TIMZON": "120",
                    "TIME": "120",
                    "RLY": "x1000",
                    "SVC": "OFF",
                    "DNTSTP": "OFF",
                    "FEATR": "OFF"
                }
            },
            {
                "objnam": "B1001",
                "params": {
                    "OBJTYP": "BODY",
                    "SUBTYP": "SPA",
                    "SNAME": "Private Spa",
                    "LISTORD": "2",
                    "HITMP": "82",
                    "LOTMP": "78",
                    "HTSRC": "H0002",
                    "SHARE": "B1000",
                    "PRIME": "C1002",
                    "SEC": "C1003",
                    "ACT1": "C1005",
                    "ACT2": "C1008",
                    "ACT3": "C1012",
                    "ACT4": "C1A3B",
                    "VOL": "",
                    "MANUAL": "OFF",
                    "MANHT": ""
                }
            },
            {
                "objnam": "H1001",
                "params": {
                    "SUBTYP": "SOLAR",
                    "SNAME": "Solar Prefered",
                    "DLY": "100",
                    "COMUART": "2",
                    "LISTORD": "1",
                    "ACT": "PRIOR"
                }
            }
        ]
    }
}

```

```
{
  "objnam": "H1002",
  "params": {
    "SUBTYP": "HCOMBO",
    "SNAME": "Solar Prefered",
    "DLY": "100",
    "COMUART": "2",
    "LISTORD": "1",
    "ACT": "PRIOR",
    "OBJLIST": [
      {
        "objnam": "h1001",
        "params": {
          "PARENT": "H1002",
          "USE": "H1000",
          "LISTORD": "2"
        }
      },
      {
        "objnam": "h1001",
        "params": {
          "PARENT": "H1002",
          "USE": "H1001",
          "LISTORD": "1"
        }
      }
    ]
  }
},
{
  "objnam": "P1001",
  "params": {
    "SUBTYP": "SINGLE / DUAL / SPEED / FLOW",
    "SNAME": "RETURN",
    "COMUART": "1",
    "PRIMFLO": "2000",
    "PRIMTIM": "2",
    "SYSTIM": "3",
    "MIN": "300",
    "MAX": "30000",
    "PRIOR": "SPEED | RECENT | CUSTOM",
    "BODY": "B1000",
    "SHARE": "B1002",
    "DLY": "ON",
    "SETTMP": "2",
    "OBJLIST": [
      {
        "objnam": "p1002",
        "params": {
          "CIRUIT": "C1010",
          "LISTORD": "1",
          "SPEED": "100",
          "BOOST": "OFF",
          "SELECT": "GPM"
        }
      }
    ]
  }
},
{
  "objnam": "V1001",
  "params": {
    "SUBTYP": "LEGACY / INTELLI",
    "SNAME": "RETURN",
    "COMUART": "1",
    "POSIT": "2",
    "ASSIGN": "RETURN",
    "DLY": "ON",
    "OBJLIST": [
      {
        "objnam": "v1001",
        "params": {
          "POSIT": "2",
          "PRIOR": "1",
          "CIRUIT": "C1010"
        }
      }
    ]
  }
}
]
```

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

Get Remotes

The GetRemotes query returns all existing Remote objects defined on the THW along with their assigned button objects. Parameters for the remote are optional in some cases based on the remote type, the THW may include a sparse set of parameters based on those it has a valid value for, optionally parameters may be included with null (0000) values. All button objects will be returned for each remote along with their parent assignment, circuit and list order parameters are optional and may be omitted if unassigned.

Example (list will include multiple remotes based on THW configuration):

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetRemotes", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetRemotes", "response": "200", "answer": [{ "objnam": "R1000", "params": { "SUBTYP": "IS4", "SNAME": "Pool Remote", "COMUART": "1", "MODULE": "1", "LISTORD": "1", "BODY": "B1000", "PUMP": "P1000", "ENABLE": "ON", "MODE": "ON", "BUTTONS": [{ "OBJNAM": "b1000", "LISTORD": "1", "PARENT": "R1000", "CIRCUIT": "C1200" }, { "OBJNAM": "b1002", "LISTORD": "2", "PARENT": "R1000", "CIRCUIT": "C1202" }, { "OBJNAM": "b1003", "LISTORD": "3", "PARENT": "R1000", "CIRCUIT": "00000" }, { "OBJNAM": "b1004", "LISTORD": "4", "PARENT": "R1000", "CIRCUIT": "00000" }] } }] }</pre>

Address Map

Created by Jim Hewitt, last modified on Jan 13, 2016

The AddressMap query returns the address assignments available for the designated object, valid objects include any object that contains a COMUART property that is programmable. The resulting set can be used to populate the Address Select drop-down. Addresses that are already assigned to devices will not be included in the result list. Valid object types include ICHEM, ICHLOR, HEATER, VALVE, PUMP and REMOTE. The THW will determine what addresses remain available for the object whose OBJNAM is contained as the arguments value based on the objects type and module assignment.

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "AddressMap", "arguments": { "OBJTYP": "CHEM", "SUBTYP": "ICHEM" } }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "AddressMap", "response": "200", "answer": [{ "type": "ICHEM", "assignments": ["1", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15", "16"] }] }</pre>

Valid types / subtypes

Type	Subtype
CHEM	ICHEM
CHEM	ICHLOR
HEATER	ULTRA
VALVE	INTELLI
PUMP	SINGLE
PUMP	DUAL
PUMP	SPEED
PUMP	FLOW
REMOTE	IS4
REMOTE	IS10
REMOTE	SPACND

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

GetStatusMessageFlags

Command to retrieve the status message flags for all objects. Status message flags control whether a status message gets created for the given condition, see table below.

Command	Response
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetStatusMessageFlags", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetStatusMessageFlags", "response": "200", "answer": { "CIRCUIT": "fhlmv", "PUMP": "svioubpawfe", "ICHLOR": "dvifloce", "ICHEM": "acbd fgijklnpehomqrs", "HEATER": "hlwrosftbpg" } }</pre>

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

Status Message Definitions:

Status Message Type (#VALUE)	Status Message ID (#value)	Application Type	Application SubType	Object name in command	Message Text	Status Message Display Flags	Notes
StatINFORM(2)	tVALVE(0xFA58)	Circuit	VALVE	VALVE	"Valve Delay : %TIMOUT sec (sName)"	v	Where %TIMEOUT is the valve delay and sName is the valve description like INTAKE or RETURN.
StatINFORM(2)	tHEATER(0xA440)	Circuit	HEATER	HEATER	"Cool Down : %TIMOUT sec"	h	Where %TIMEOUT is the time before the pool pump is turned off and heater is turned off (initially in DLYON).
StatWARNNG(1)	tFRZ(0xA114)	Circuit	FRZ	FRZ	"Freezing Active"	f	Freeze protection is ON.
StatINFORM(2)	tLIGHT(0xAF94)	Circuit	LIGHT	LIGHT	"Light Delay : %TIMOUT sec"	l	Where %TIMOUT is the delay before the light group sequence starts.
StatINFORM(2)	tMASTER(0xB6D4)	Circuit	MASTER	MASTER	"Priming : %TIMOUT sec"	m	Where %TIMOUT is the Pool Priming time before

							the Master Cleaner turns on.
StatINFORM(2)	tPUMP(0xD68C)	Pump	N/A	PUMP	"%RPM RPM, %PWR Watts" "%RMP RMP, %GPM GPM, %PWR Watts"	s	If IntelliFlowVF then the 1stform is used. If IntelliFlowVS then the 2ndform is used. %RPM is filled with the speed, %GPM is filled with the flow (gal per minute) and %PWR is filled with the power in Watts.
StatALERT(0)	tVOLT(0xFD10)	Pump	N/A	PUMP	"Over Voltage Alarm"	v	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tCURRENT(0x9464)	Pump	N/A	PUMP	"Over Current Alarm"	i	
StatALERT(0)	tFAULT(0x9DBE)	Pump	N/A	PUMP	"Power Outage Alarm"	o	
StatALERT(0)	tALARM(0x835E)	Pump	N/A	PUMP	"Unknown Alarm"	u	
StatALERT(0)	tSYSTEM(0xF1BC)	Pump	N/A	PUMP	"Pump Override"	b	
StatALERT(0)	tPRIM(0xD448)	Pump	N/A	PUMP	"Priming Alarm"	p	
StatALERT(0)	tDRVALM(0x9B75)	Pump	N/A	PUMP	"Drive Temperature Alarm"	a	
StatWARNNG(1)	tDRVWRN(0x9B76)	Pump	N/A	PUMP	"Drive Temperature Warning"	w	
StatWARNNG(1)	tFILTER(0x9E5C)	Pump	N/A	PUMP	"Filter Warning"	f	
StatWARNNG(1)	tCOMERR(0x9172)	Pump	N/A	PUMP	"Communication Lost"	e	
StatALERT(0)	tDEVICE(0x971C)	IntelliChlor	N/A	INTELLICHLOR	"PCB Error"	d	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tVOLT(0xFD10)	IntelliChlor	N/A	INTELLICHLOR	"Low Voltage Alarm"	v	
StatALERT(0)	tCURRENT(0x9464)	IntelliChlor	N/A	INTELLICHLOR	"High Current Alarm"	i	
StatALERT(0)	tCUTOFF(0x9512)	IntelliChlor	N/A	INTELLICHLOR	"Cold Water Cutoff Alarm"	f	
StatWARNNG(1)	tSALTLO(0xE084)	IntelliChlor	N/A	INTELLICHLOR	"Low Salt Warning"	l	
StatWARNNG(1)	tVERYLO(0xFBB4)	IntelliChlor	N/A	INTELLICHLOR	"Very Low Salt Warning"	o	
StatWARNNG(1)	tCLEAN(0x8FA2)	IntelliChlor	N/A	INTELLICHLOR	"Clean and Temperature Alarm"	c	

					Inspect Alarm		
StatWARNNG(1)	tCOMERR(0x9172)	IntelliChlor	N/A	INTELLICHLOR	"Communication Lost"	e	
StatALERT(0)	tNOFLO(0xBD2C)	IntelliChem	N/A	INTELLICHEM	"No Flow"	a	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tPHHI(0xCC20)	IntelliChem	N/A	INTELLICHEM	"pH Too High"	c	
StatALERT(0)	tPHLO(0xCD08)	IntelliChem	N/A	INTELLICHEM	"pH Too Low"	b	
StatALERT(0)	tORPLO(0xC3F8)	IntelliChem	N/A	INTELLICHEM	"ORP Too Low"	d	
StatALERT(0)	tPHCHK(0xCB38)	IntelliChem	N/A	INTELLICHEM	"Check pH Chemical Container"	f	
StatALERT(0)	tORPCHK(0xC228)	IntelliChem	N/A	INTELLICHEM	"Check ORP Chemical Container"	g	
StatALERT(0)	tLOCKOUT(0xB2C0)	IntelliChem	N/A	INTELLICHEM	"Sanitizer Locked Out"	i	
StatALERT(0)	tPHLIM(0xCC94)	IntelliChem	N/A	INTELLICHEM	"pH at Feed Limit"	j	
StatALERT(0)	tORPLIM(0xC384)	IntelliChem	N/A	INTELLICHEM	"ORP at Feed Limit"	k	
StatALERT(0)	tINVALID(0xACDC)	IntelliChem	N/A	INTELLICHEM	"Invalid Settings"	l	
StatALERT(0)	tCALIB(0x8D24)	IntelliChem	N/A	INTELLICHEM	"Calibration Failed"	n	
StatALERT(0)	tCOMERR(0x9172)	IntelliChem	N/A	INTELLICHEM	"Automation Comm Error"	p	
StatWARNNG(1)	tORPHI(0xC310)	IntelliChem	N/A	INTELLICHEM	"ORP Too High"	e	
StatWARNNG(1)	tPROBE(0xD618)	IntelliChem	N/A	INTELLICHEM	"Probe Check Failed"	h	
StatWARNNG(1)	tCOMLNK(0x9294)	IntelliChem	N/A	INTELLICHEM	"Peripheral Comm Error"	o	
StatWARNNG(1)	tDEVICE(0x971C)	IntelliChem	N/A	INTELLICHEM	"Memory Errors"	m	
StatINFORM(2)	tFLOWDLY (0x9FB8)	IntelliChem	N/A	INTELLICHEM	"Flow Delay ON"	q	
StatINFORM(2)	tPHMOD(0xCD7C)	IntelliChem	N/A	INTELLICHEM	"pH Mode is %PHMOD"	r	Where %PHMOD is filled with the PH Mode string {DOSE, MIX, MONITOR}
StatINFORM(2)	tORPMOD(0xC46C)	IntelliChem	N/A	INTELLICHEM	"ORP Mode is %ORPMOD"	s	Where %ORPMOD is filled with the ORP Mode string {DOSE, MIX, MONITOR}
StatALERT(0)	tHITMP(0xA4D2)	HeaterUltraT	N/A	HEATERULTRAT	"High Water Temperature."	h	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tLOTMP(0xB456)	HeaterUltraT	N/A	HEATERULTRAT	"Low Water Temperature."	l	
StatALERT(0)	tFLOW(0x9F44)	HeaterUltraT	N/A	HEATERULTRAT	"Low Water Flow."	w	

					Flow.		
StatALERT(0)	tRLY(0xDBFC)	HeaterUltraT	N/A	HEATERULTRAT	"Remote Relay."	r	
StatALERT(0)	tOFF(0xC058)	HeaterUltraT	N/A	HEATERULTRAT	"Water Thermistor open."	o	
StatALERT(0)	tON(0xC140)	HeaterUltraT	N/A	HEATERULTRAT	"Water Thermistor shorted."	s	
StatALERT(0)	tOVROFF(0xC7BE)	HeaterUltraT	N/A	HEATERULTRAT	"Defrost Thermistor open."	f	
StatALERT(0)	tOVRON(0xC7E4)	HeaterUltraT	N/A	HEATERULTRAT	"Defrost Thermistor shorted."	t	
StatALERT(0)	tOUT(0xC798)	HeaterUltraT	N/A	HEATERULTRAT	"Brownout."	b	
StatALERT(0)	tHI(0xA4B4)	HeaterUltraT	N/A	HEATERULTRAT	"High refrigerant pressure."	p	
StatALERT(0)	tLO(0xB1D8)	HeaterUltraT	N/A	HEATERULTRAT	"Low refrigerant pressure."	g	

GetSystemConfiguration

The GetConfiguration query returns all configuration (Saved / Hard Coded) data necessary to display the home page. This includes all bodies, lights, light groups, features, heaters, and chemistry. The client will use this information to subscribe to Live parameters based on the screen being displayed. Items that are not included in this message include any circuits that are displayed only in configuration screens, orphaned heaters, chemistry and circuit affects.

Changes:

- 1. The PARENT property was added to all CIRCGRP and LITSHO objects
- 2. The CHILD property was added to all CIRCUIT objects
- 3. A Heater Combo has been added to the body circuit for clarification

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetSystemConfiguration", "arguments": "" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetSystemConfiguration", "response": "200", "answer": [{ "objnam": "UFFFFE", "params": { "ENABLE": "ON" } }, { "objnam": "_C10C", "params": { "CLK24A": "AMPMP" } }, { "objnam": "_5451", "params": { "MODE": "ENGLISH" } }, { "objnam": "_FEA2", "params": { "STATUS": "ON", "SUBTYP": "FRZ", "OBJTYP": "CIRCUIT" } }, { "objnam": "B0001", "params": { "OBJTYP": "BODY", "SUBTYP": "POOL", "SNAME": "Lower Pool", "LISTORD": "1", "HITMP": "82", "LOTMP": "78", "HTSRC": "H0001", "SHARE": "B1002", "PRIME": "C1002", "SEC": "C1003", "ACT1": "C1005", "ACT2": "C1008", "ACT3": "C1012", "ACT4": "C1A3B", "OBJLIST": [{ "objnam": "H0001", "params": { "OBJTYP": "HEATER", "SUBTYP": "HTPMP", "SNAME": "Heat Pump",</pre>

```

        "LISTORD": "1"
    }
},
{
    "objnam": "H0002",
    "params": {
        "OBJTYP": "HEATER",
        "SUBTYP": "SOLAR",
        "SNAME": "SolarPanel",
        "LISTORD": "2"
    }
},
{
    "objnam": "H0002",
    "params": {
        "OBJTYP": "HCOMBO",
        "SUBTYP": "SOLAR",
        "SNAME": "Solar preferred",
        "LISTORD": "3",
        "OBJLIST": [
            {
                "objnam": "H0002",
                "params": {
                    "OBJTYP": "HEATER",
                    "SUBTYP": "SOLAR",
                    "SNAME": "SolarPanel",
                    "LISTORD": "1"
                }
            },
            {
                "objnam": "H0001",
                "params": {
                    "OBJTYP": "HEATER",
                    "SUBTYP": "HTPMP",
                    "SNAME": "Heat Pump",
                    "LISTORD": "2"
                }
            }
        ]
    }
},
{
    "objnam": "I1000",
    "params": {
        "OBJTYP": "CHEM",
        "SUBTYP": "ICHLOR",
        "SNAME": "PoolClorinator"
    }
},
{
    "objnam": "I1001",
    "params": {
        "OBJTYP": "CHEM",
        "SUBTYP": "ICHEM",
        "SNAME": "Chemistry"
    }
}
]
},
{
    "objnam": "R1000",
    "params": {
        "SUBTYP": "IS4",
        "SNAME": "Pool Remote",
        "COMUART": "1",
        "MODULE": "1",
        "LISTORD": "1",
        "BODY": "B1000",
        "PUMP": "P1000",
        "ENABLE": "ON",
        "MODE": "ON",
        "BUTTONS": [
            {
                "OBJNAM": "b1000",
                "LISTORD": "1",
                "PARENT": "R1000",
                "CIRCUIT": "C1200"
            },
            {
                "OBJNAM": "b1002",
                "LISTORD": "2",
                "PARENT": "R1000",
                "CIRCUIT": "C1202"
            }
        ]
    }
},

```

```

        {
            "OBJNAM": "b1003",
            "LISTORD": "3",
            "PARENT": "R1000",
            "CIRCUIT": "00000"
        },
        {
            "OBJNAM": "b1004",
            "LISTORD": "4",
            "PARENT": "R1000",
            "CIRCUIT": "00000"
        }
    ]
},
{
    "objnam": "C1002",
    "params": {
        "OBJTYP": "CIRCUIT",
        "SUBTYP": "MAGIC2",
        "SNAME": "PoolLight",
        "LISTORD": "1",
        "SHOMNU": "fl",
        "CHILD": "c1001, c1002",
        "USE": "OFF",
        "USAGE": "RPCSqDYmBAsOWGitEIgNQbarl",
        "OBJLIST": [
            {
                "objnam": "c1000",
                "params": {
                    "OBJTYP": "CIRCGRP",
                    "CIRCUIT": "C1002",
                    "PARENT": "C1000",
                    "ACT": "OFF",
                    "LISTORD": "2",
                    "DLY": "5"
                }
            },
            {
                "objnam": "c1002",
                "params": {
                    "OBJTYP": "LITSHO",
                    "CIRCUIT": "C1002",
                    "PARENT": "C1000",
                    "ACT": "OFF",
                    "LISTORD": "2",
                    "DLY": "10"
                }
            }
        ]
    }
}
]
}

```

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

 Like Be the first to like this

No labels 

GetUserSettings

The GetUserSettings query returns information about the property owner, the property location, date/time settings and user preferences.

GetQuery	SendQuery
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetUserSettings", "arguments": "" }</pre>	

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

Parameters returned:

GetValidLightGroupings

Command to retrieve all light circuits that can be grouped together.

Command	Response
<pre>{ "command": "GetQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetValidLightGroupings", "arguments": "C1200" }</pre>	<pre>{ "command": "SendQuery", "messageID": "10A86ef1-88f1-8091-7cdf1b", "queryName": "GetValidLightGroupings", "response": "200", "answer": [{ "objnam": "C1200", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "INTELLI", "SNAME": "East Pool 1" } }, { "objnam": "C1201", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "INTELLI", "SNAME": "East Pool 2" } }, { "objnam": "C1202", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "INTELLI", "SNAME": "East Pool 3" } }, { "objnam": "C1203", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "INTELLI", "SNAME": "West Pool 1" } }, { "objnam": "C1204", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "SAML", "SNAME": "West Pool 2" } }, { "objnam": "C1205", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "PHOTON", "SNAME": "West Pool 3" } }, { "objnam": "C1206", "params": { "OBJTYP": "CIRCUIT", "SUBTYP": "MAGIC2", "SNAME": "Spa" } }] }</pre>

Response Codes:

Code	Definition
200	OK
400	Bad request, unable to process message contents.

MasterOff

The MasterOff command turns off all circuits with the exception of delays, overrides and Freeze conditions. This command will be issued to the THW when the user operates the All Off button on the home page and has the intent of turning off everything that can be turned on on the property.

Example:

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "MasterOff" }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "MasterOff", "response": "200" }</pre>

Response Codes:

Code	Definition
200	OK, A status message may be generated if the action results in a DLYON or DLYOFF state
400	Bad request, unable to process message contents.
403	Forbidden for any reason, eg: service mode, etc.

SetBodyState

SetBodyState turns the body of water on or off.

Example:

SetCommand	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetBodyState", "arguments": { "B1000": "ON" } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetBodyState", "response": "200" }</pre>

Response Codes:

Code	Definition
200	OK, A status message may be generated if the action results in a DLYON or DLYOFF state
304	Not modified, body already in requested state.
400	Bad request, unable to process message contents.
403	Forbidden for any reason, eg: service mode, etc.
404	Not found, the objnam (key) was not found.

SetStatusMessageFlags

Command to set the status message flags for a given object type. Status message flags control whether a status message gets created for the given condition, see table below. A list of one or more key / value pairs can be included in this message.

Command	Response
<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetStatusMessageFlags", "arguments": { "CIRCUIT": "fhlmv", "PUMP": "svioubpawfe", "ICHLOR": "dvifloce", "ICHEM": "acbd fgijklnpehomqrs", "HEATER": "hlwrosftbpg" } }</pre>	<pre>{ "command": "SetCommand", "messageID": "10A86ef1-88f1-8091-7cdf2b", "method": "SetStatusMessageFlags", "response": "200" }</pre>

Response Codes:

Code	Definition
200	OK, Operation successful
400	Bad request, unable to process message contents.

Status Message Definitions:

Status Message Type (#VALUE)	Status Message ID (#value)	Application Type	Application SubType	Object name in command	Message Text	Status Message Display Flags	Notes
StatINFORM(2)	tVALVE(0xFA58)	Circuit	VALVE	VALVE	"Valve Delay : %TIMEOUT sec (sName)"	v	Where %TIMEOUT is the valve delay and sName is the valve description like INTAKE or RETURN.
StatINFORM(2)	tHEATER(0xA440)	Circuit	HEATER	HEATER	"Cool Down : %TIMEOUT sec"	h	Where %TIMEOUT is the time before the pool pump is turned off and heater is turned off (initially in DLYON).
StatWARNNG(1)	tFRZ(0xA114)	Circuit	FRZ	FRZ	"Freezing Active"	f	Freeze protection is ON.
StatINFORM(2)	tLIGHT(0xAF94)	Circuit	LIGHT	LIGHT	"Light Delay : %TIMEOUT sec"	l	Where %TIMEOUT is the delay before the light group sequence starts.
StatINFORM(2)	tMASTER(0xB6D4)	Circuit	MASTER	MASTER	"Priming : %TIMEOUT sec"	m	Where %TIMEOUT is the Pool Priming time before the Master

							Cleaner turns on.
StatINFORM(2)	tPUMP(0xD68C)	Pump	N/A	PUMP	"%RPM RPM, %PWR Watts" "%RMP RMP, %GPM GPM, %PWR Watts"	s	If IntelliFlowVF then the 1stform is used. If IntelliFlowVS then the 2ndform is used. %RPM is filled with the speed, %GPM is filled with the flow (gal per minute) and %PWR is filled with the power in Watts.
StatALERT(0)	tVOLT(0xFD10)	Pump	N/A	PUMP	"Over Voltage Alarm"	v	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tCURRENT(0x9464)	Pump	N/A	PUMP	"Over Current Alarm"	i	
StatALERT(0)	tFAULT(0x9DBE)	Pump	N/A	PUMP	"Power Outage Alarm"	o	
StatALERT(0)	tALARM(0x835E)	Pump	N/A	PUMP	"Unknown Alarm"	u	
StatALERT(0)	tSYSTEM(0xF1BC)	Pump	N/A	PUMP	"Pump Override"	b	
StatALERT(0)	tPRIM(0xD448)	Pump	N/A	PUMP	"Priming Alarm"	p	
StatALERT(0)	tDRVALM(0x9B75)	Pump	N/A	PUMP	"Drive Temperature Alarm"	a	
StatWARNNG(1)	tDRVWRN(0x9B76)	Pump	N/A	PUMP	"Drive Temperature Warning"	w	
StatWARNNG(1)	tFILTER(0x9E5C)	Pump	N/A	PUMP	"Filter Warning"	f	
StatWARNNG(1)	tCOMERR(0x9172)	Pump	N/A	PUMP	"Communication Lost"	e	
StatALERT(0)	tDEVICE(0x971C)	IntelliChlor	N/A	INTELLICHLOR	"PCB Error"	d	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tVOLT(0xFD10)	IntelliChlor	N/A	INTELLICHLOR	"Low Voltage Alarm"	v	
StatALERT(0)	tCURRENT(0x9464)	IntelliChlor	N/A	INTELLICHLOR	"High Current Alarm"	i	
StatALERT(0)	tCUTOFF(0x9512)	IntelliChlor	N/A	INTELLICHLOR	"Cold Water Cutoff Alarm"	f	
StatWARNNG(1)	tSALTLO(0xE084)	IntelliChlor	N/A	INTELLICHLOR	"Low Salt Warning"	l	
StatWARNNG(1)	tVERYLO(0xFBB4)	IntelliChlor	N/A	INTELLICHLOR	"Very Low Salt Warning"	o	
StatWARNNG(1)	tCLEAN(0x8FA2)	IntelliChlor	N/A	INTELLICHLOR	"Clean and Temperature Alarm"	c	

					Inspect Alarm"		
StatWARNNG(1)	tCOMERR(0x9172)	IntelliChlor	N/A	INTELLICHLOR	"Communication Lost"	e	
StatALERT(0)	tNOFLO(0xBD2C)	IntelliChem	N/A	INTELLICHEM	"No Flow"	a	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tPHHI(0xCC20)	IntelliChem	N/A	INTELLICHEM	"pH Too High"	c	
StatALERT(0)	tPHLO(0xCD08)	IntelliChem	N/A	INTELLICHEM	"pH Too Low"	b	
StatALERT(0)	tORPLO(0xC3F8)	IntelliChem	N/A	INTELLICHEM	"ORP Too Low"	d	
StatALERT(0)	tPHCHK(0xCB38)	IntelliChem	N/A	INTELLICHEM	"Check pH Chemical Container"	f	
StatALERT(0)	tORPCHK(0xC228)	IntelliChem	N/A	INTELLICHEM	"Check ORP Chemical Container"	g	
StatALERT(0)	tLOCKOUT(0xB2C0)	IntelliChem	N/A	INTELLICHEM	"Sanitizer Locked Out"	i	
StatALERT(0)	tPHLIM(0xCC94)	IntelliChem	N/A	INTELLICHEM	"pH at Feed Limit"	j	
StatALERT(0)	tORPLIM(0xC384)	IntelliChem	N/A	INTELLICHEM	"ORP at Feed Limit"	k	
StatALERT(0)	tINVALID(0xACDC)	IntelliChem	N/A	INTELLICHEM	"Invalid Settings"	l	
StatALERT(0)	tCALIB(0x8D24)	IntelliChem	N/A	INTELLICHEM	"Calibration Failed"	n	
StatALERT(0)	tCOMERR(0x9172)	IntelliChem	N/A	INTELLICHEM	"Automation Comm Error"	p	
StatWARNNG(1)	tORPHI(0xC310)	IntelliChem	N/A	INTELLICHEM	"ORP Too High"	e	
StatWARNNG(1)	tPROBE(0xD618)	IntelliChem	N/A	INTELLICHEM	"Probe Check Failed"	h	
StatWARNNG(1)	tCOMLNK(0x9294)	IntelliChem	N/A	INTELLICHEM	"Peripheral Comm Error"	o	
StatWARNNG(1)	tDEVICE(0x971C)	IntelliChem	N/A	INTELLICHEM	"Memory Errors"	m	
StatINFORM(2)	tFLOWDLY (0x9FB8)	IntelliChem	N/A	INTELLICHEM	"Flow Delay ON"	q	
StatINFORM(2)	tPHMOD(0xCD7C)	IntelliChem	N/A	INTELLICHEM	"pH Mode is %PHMOD"	r	Where %PHMOD is filled with the PH Mode string {DOSE, MIX, MONITOR}
StatINFORM(2)	tORPMOD(0xC46C)	IntelliChem	N/A	INTELLICHEM	"ORP Mode is %ORPMOD"	s	Where %ORPMOD is filled with the ORP Mode string {DOSE, MIX, MONITOR}
StatALERT(0)	tHITMP(0xA4D2)	HeaterUltraT	N/A	HEATERULTRAT	"High Water Temperature."	h	Status Message flags are stored in a string which may be set/get with parameter tMASKSM
StatALERT(0)	tLOTMP(0xB456)	HeaterUltraT	N/A	HEATERULTRAT	"Low Water Temperature."	l	
StatALERT(0)	tFLOW(0x9F44)	HeaterUltraT	N/A	HEATERULTRAT	"Low Water Flow."	w	

					Flow.		
StatALERT(0)	tRLY(0xDBFC)	HeaterUltraT	N/A	HEATERULTRAT	"Remote Relay."	r	
StatALERT(0)	tOFF(0xC058)	HeaterUltraT	N/A	HEATERULTRAT	"Water Thermistor open."	o	
StatALERT(0)	tON(0xC140)	HeaterUltraT	N/A	HEATERULTRAT	"Water Thermistor shorted."	s	
StatALERT(0)	tOVROFF(0xC7BE)	HeaterUltraT	N/A	HEATERULTRAT	"Defrost Thermistor open."	f	
StatALERT(0)	tOVRON(0xC7E4)	HeaterUltraT	N/A	HEATERULTRAT	"Defrost Thermistor shorted."	t	
StatALERT(0)	tOUT(0xC798)	HeaterUltraT	N/A	HEATERULTRAT	"Brownout."	b	
StatALERT(0)	tHI(0xA4B4)	HeaterUltraT	N/A	HEATERULTRAT	"High refrigerant pressure."	p	
StatALERT(0)	tLO(0xB1D8)	HeaterUltraT	N/A	HEATERULTRAT	"Low refrigerant pressure."	g	